1장 도커란 무엇인가?

1절 도커란 무엇인가?

1.도커의 정체

- 도커 정의
  - "데이터 또는 프로그램을 각각 독립된 환경에 격리시키는 기능"을 제공하는 소프트웨어
  - 운영체제(유사) 통째로 격리
  - 주로 서버에서 사용

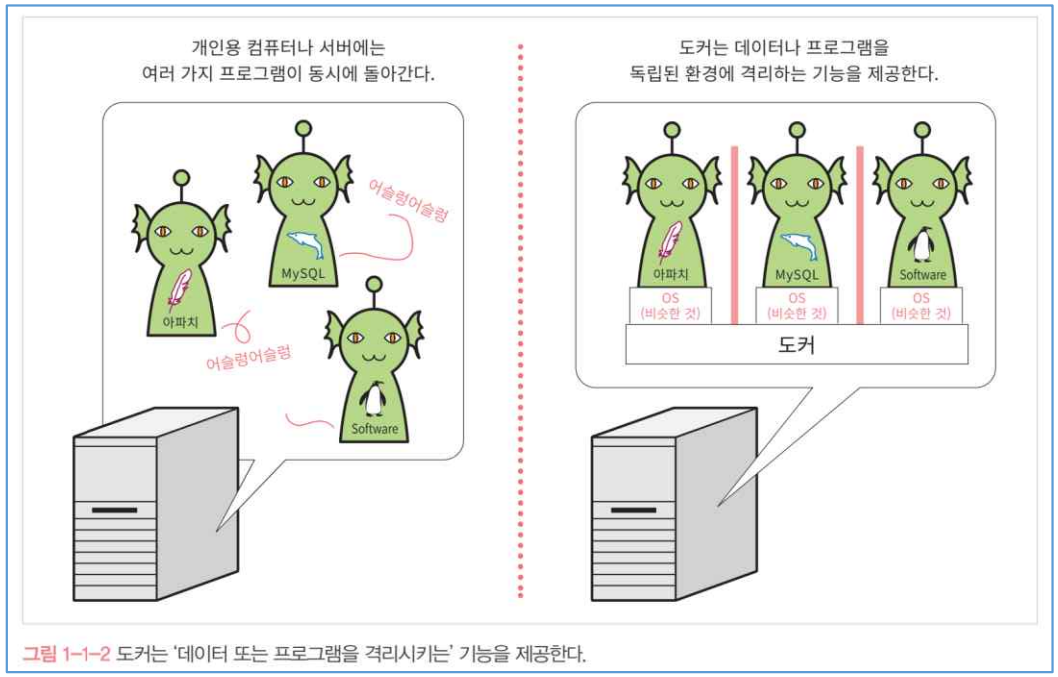


그림 1-1-2 도커는 '데이터 또는 프로그램을 격리시키는' 기능을 제공한다.

1.도커의 정체

- 도커란? (레드햇 기술문서 중)

Docker 기술은 Linux 커널과 Cgroups 및 네임스페이스 등 커널의 기능을 사용하여 프로세스를 분리함으로써 독립적으로 실행할 수 있도록 합니다. 이러한 독립성은 컨테이너의 본래 목적입니다. 다시 말해서, 여러 프로세스와 애플리케이션을 서로 개별적으로 실행하여 인프라를 더 효과적으로 활용하고 개별 시스템을 사용할 때와 동일한 보안을 유지할 수 있습니다.

<출처> https://www.redhat.com/ko/topics/containers/what-is-docker

- 컨테이너 기반 기술(Isolation)
- 리눅스 커널에서 제공하는 기능
- 하나의 시스템에서 프로세스를 격리 시킬 수 있는 가상화 기술

<리눅스 namespace 관련 주요 명령>
https://manpages.ubuntu.com/manpages/xenial/man1/unshare.1.html
https://manpages.ubuntu.com/manpages/xenial/en/man1/nsenter.1.html

1.도커의 정체

- 컨테이너와 도커 엔진

  컨테이너를 다루는 기능을 제공하는 소프트웨어 – 도커

- 도커 엔진 설치 필요

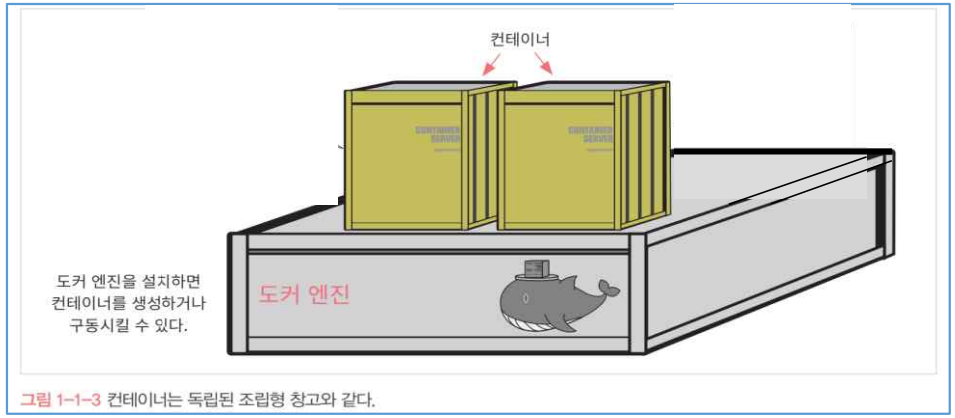


그림 1-1-3 컨테이너는 독립된 조립형 창고와 같다.

- 컨테이너 생성을 위해 이미지 필요

아파치 이미지
MySQL 이미지
. . .

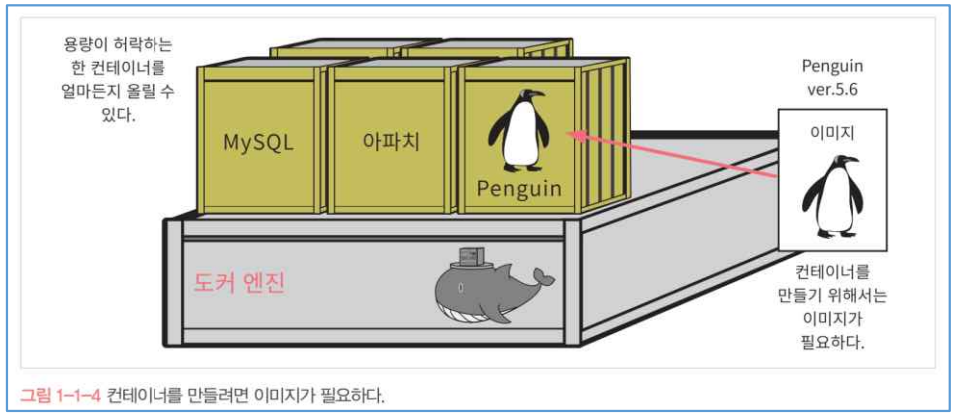


그림 1-1-4 컨테이너를 만들려면 이미지가 필요하다.

1.도커의 정체

- 도커는 리눅스에서!
  - 도커는 리눅스 운영체제 필요
  - 도커는 리눅스 기반을 전제로 만들어졌기 때문에 다음 운영체제보다는 리눅스에서 사용하기를 권장

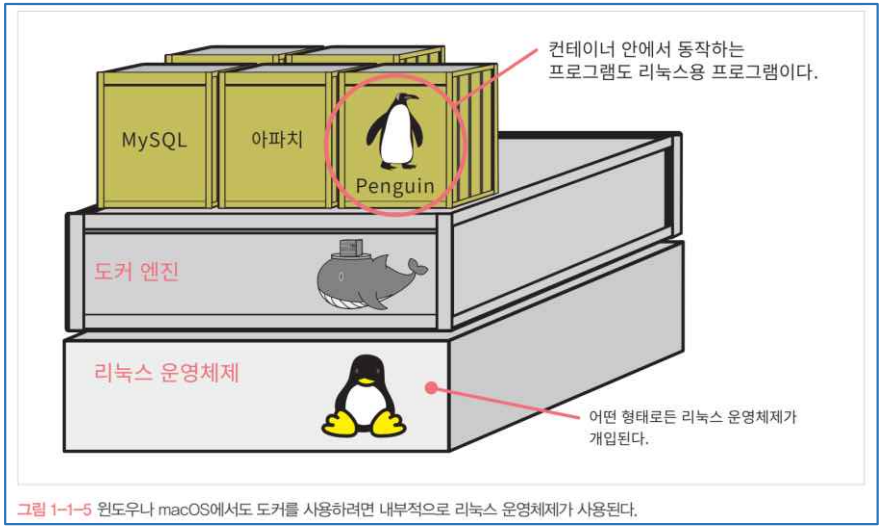


그림 1-1-5 윈도우나 macOS에서도 도커를 사용하려면 내부적으로 리눅스 운영체제가 사용된다.

2.독립된 환경이 필요한 이유

- 데이터나 프로그램을 독립된 환경에 격리해야 하는 이유
  - 프로그램이 수행될때에는 관련 환경, 라이브러리 및 다른 프로그램이 필요



그림 1-1-6 프로그램을 실행하기 위해서는 그 프로그램의 실행 환경이나 라이브러리가 필요하다.

  - 예) 워드프레스 : MySQL 필요

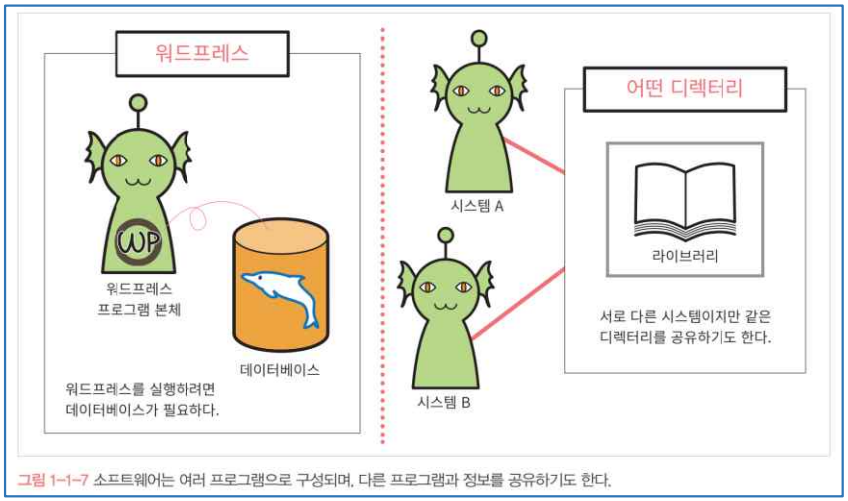


그림 1-1-7 소프트웨어는 여러 프로그램으로 구성되며, 다른 프로그램과 정보를 공유하기도 한다.

2.독립된 환경이 필요한 이유

- 특정 프로그램을 업데이트하면 다른 프로그램에도 영향을 미칠 수 있음

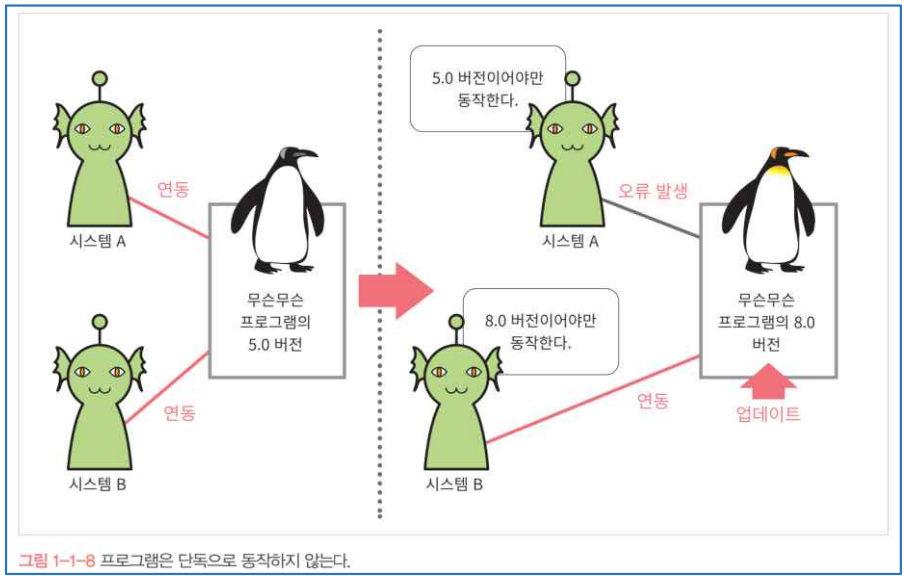


그림 1-1-8 프로그램은 단독으로 동작하지 않는다.

- 소프트웨어 업데이트 뿐만 아니라 실행환경이나 라이브러리, 디렉토리나 설정파일을 수정하는 경우가 발생할 수도 있음

- 문제의 원인은 대부분 프로그램간의 공유에 있다!

3. 프로그램의 격리

- 프로그램의 격리란?
  - 컨테이너 안의 프로그램은 다른 프로그램과 격리된 상태

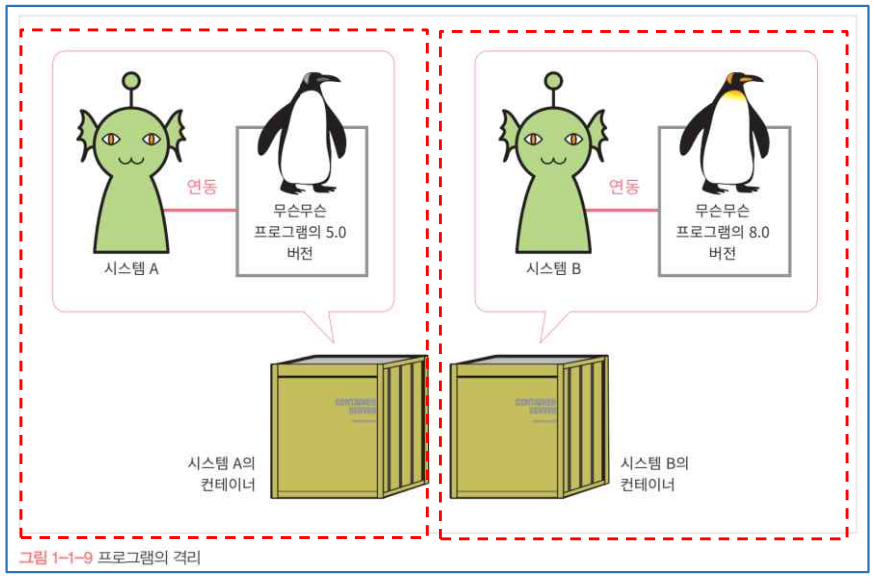


그림 1-1-9 프로그램의 격리

  - 완전히 독립된 환경이므로 동일한 버전 또는 다른 버전의 같은 프로그램 수행 가능

1장 도커란 무엇인가?

2절 서버와 도커

1.서버의 두가지 의미

- 서버란 '어떤 서비스를 제공하는 것'을 의미



그림 1-2-1 서버란 무엇인가?

- 기능적 서버와 물리적 서버



그림 1-2-2 서버의 역할

1.서버의 두가지 의미

- 서버의 기능은 소프트웨어에서 제공

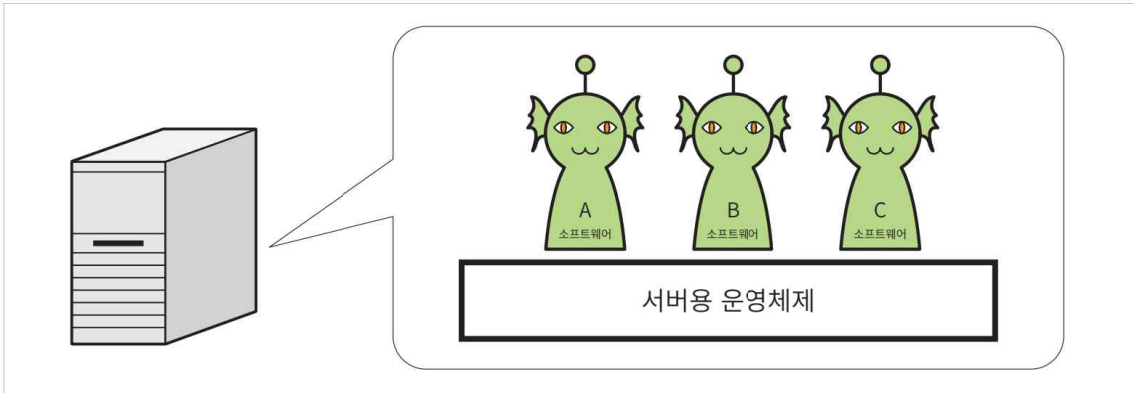


그림 1-2-6 서버 역시 운영체제가 동작하고 그 위에서 소프트웨어가 실행된다.

웹 서버용 소프트웨어를 설치 ➡ 웹 서버가 된다.
메일 서버용 소프트웨어를 설치 ➡ 메일 서버가 된다.

여러 기능적의미의 서버가 하나의 물리적 컴퓨터에 함께 존재할 수 있다 **도커의 장점과 관련!**

1.서버의 두가지 의미

• 서버의 대표적인 예

서버의 대표적인 예

서버의 종류	설명
웹 서버	웹 사이트 기능을 제공하는 서버. HTML 파일 또는 이미지 파일, 프로그램을 배치하고, 클라이언트 컴퓨터의 브라우저에서 접근해오면 이들 파일을 제공한다. 대표적인 소프트웨어로 아파치, Nginx, IIS 등이 있다.
메일 서버	메일 송수신을 담당하는 SMTP 서버와 클라이언트에 메일을 전달하는 POP 서버로 나뉜다. 이들을 합쳐 메일 서버라고 부르는 경우가 많다. 메일을 내려받지 않고 서버에 둔 채로 읽는 IMAP4 서버도 있다. 대표적인 소프트웨어로 Sendmail, Postfix, Dovecot이 있다.
데이터베이스 서버	데이터를 저장하거나 검색하는 데이터베이스 기능을 제공하는 서버다. 대표적인 소프트웨어로 MySQL, PostgreSQL, MariaDB, SQL Server, Oracle Database가 있다.
파일 서버	파일을 저장하고 다른 사람과 공유하기 위한 서버다. 대표적인 소프트웨어로 Samba가 있다.
DNS 서버	IP 주소와 도메인을 연결해주는 DNS 기능을 제공하는 서버.
DHCP 서버	IP 주소를 자동으로 할당하는 기능을 제공하는 서버.
FTP 서버	FTP 프로토콜을 사용해 파일 송수신 기능을 제공하는 서버. 웹 서버와 함께 설치하는 경우가 많으며, 파일을 제공하는 데 사용한다.
프락시 서버	통신을 중계하는 역할을 맡는 서버의 통칭. 사내 로컬 네트워크 등 인터넷과 격리된 장소에서 인터넷 상의 서버에 접속할 때 사용한다. 프락시 서버를 경유하면 접속 대상 컴퓨터에 자신의 정체를 숨길 수 있으므로 정체를 드러내고 싶지 않을 때도 사용한다.
인증 서버	사용자 인증을 위한 서버. 윈도우 네트워크에 로그인하기 위한 Active Directory 서버 또는 무선 LAN이나 원격 접속에서 사용자 인증을 맡는 Radius 서버 등이 있다. 대표적인 소프트웨어로 OpenLDAP, Active Directory가 있다.

1.서버의 두가지 의미

- 서버의 운영체제로는 주로 리눅스 사용

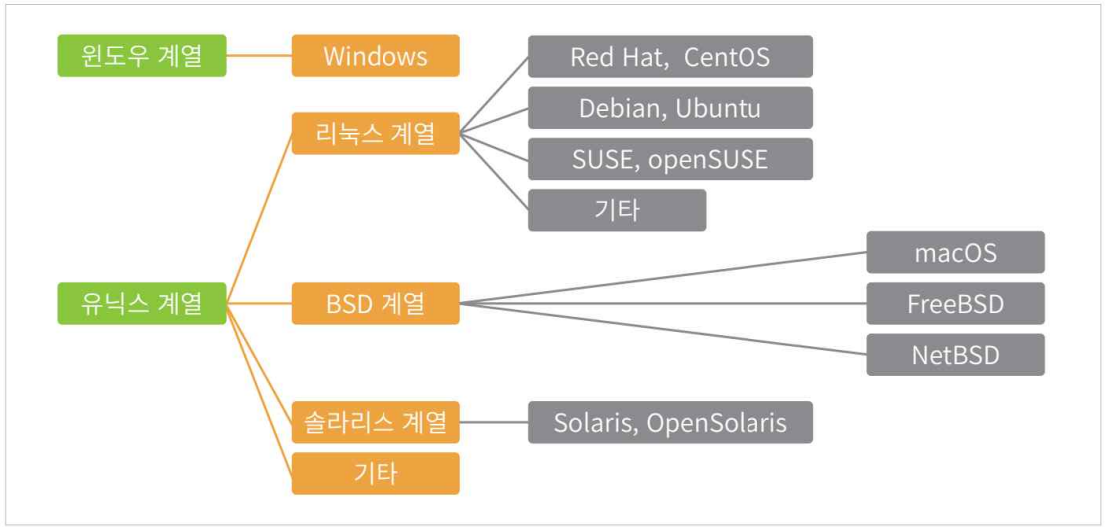


그림 1-2-7 서버 운영체제의 종류

2.여러 서버의 기능을 안전하게 실행하기

- 컨테이너 기술을 이용한 여러 서버의 안전한 실행이 가능

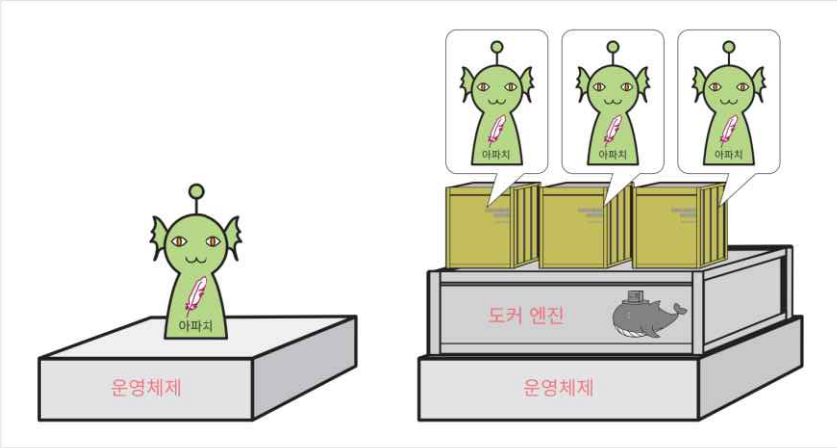


그림 1-2-8 컨테이너 기술을 활용하면 하나의 물리 서버에 여러 개의 웹 서버를 띄울 수 있다.

- 물리적인 1대의 컴퓨터에 웹서버, 메일서버, DB서버를 각 독립적인 환경에서 수행
- 1대의 서버에서 여러 개의 웹서버 구동 가능
- 물리 서버의 수를 줄일 수 있으므로 비용 절감
- 개발 환경을 갖추거나 운영환경으로 쉽게 옮길 수 있다는 특성

3.자유로이 옮길 수 있는 컨테이너

- 다른 도커 엔진으로의 이동이 용이
  - 동일한 상태로 튜닝한 컨테이너를 팀원 전체에게 배포하여 동일한 개발 환경 사용이 용이
  - 운영체제 무관, 도커만 설치되어 있으면 가능
  - 물리적 환경의 차이나 서버 구성의 차이 등 운영서버와 개발서버 사이의 환경 차이로 인한 문제를 근본적으로 방지할 수 있음

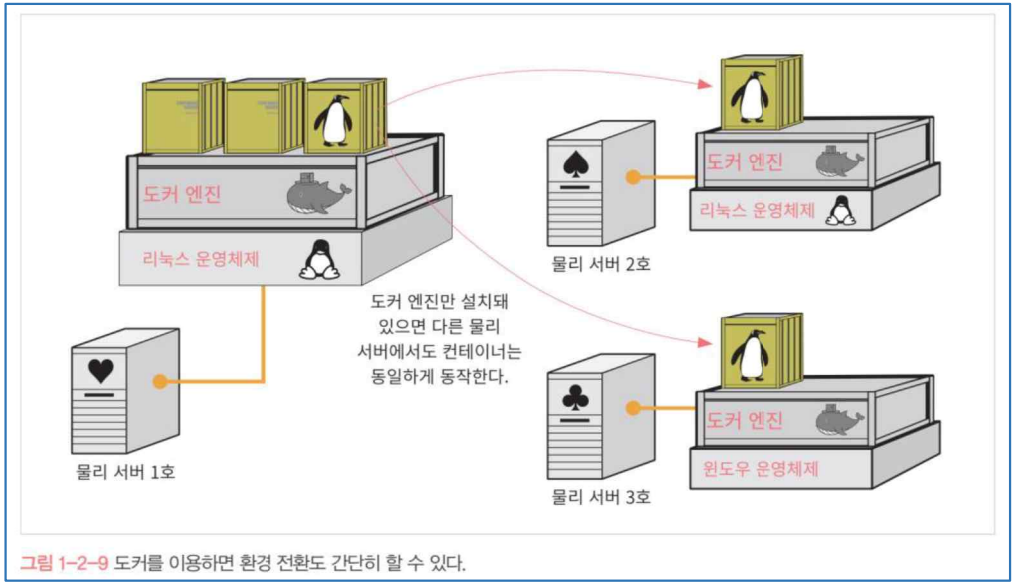


그림 1-2-9 도커를 이용하면 환경 전환도 간단히 할 수 있다.

2장 도커의 동작 원리

1절 도커의 동작원리

1.도커의 구조

• 도커의 구조

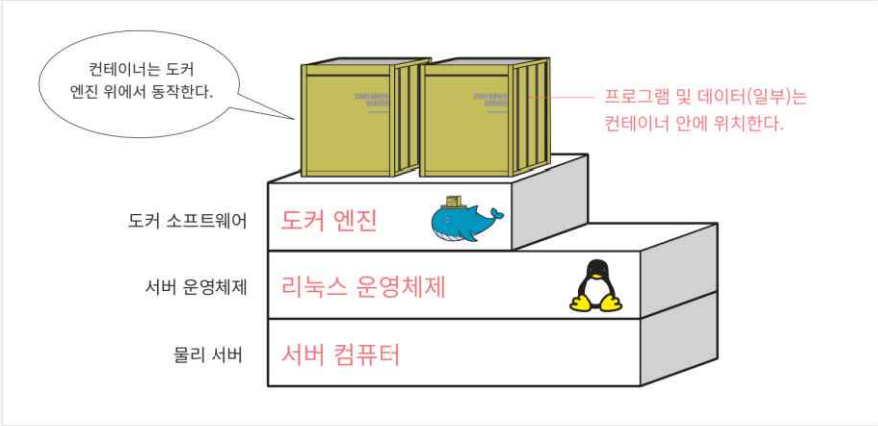


그림 2-1-1 컨테이너는 도커 엔진 위에서 동작한다.

- 운영체제 위에 도커 엔진이 동작하고 그 위에서 컨테이너가 동작
- 프로그램과 데이터는 컨테이너 안에 위치

1.도커의 구조

- 모든 컨테이너 안에는 리눅스 운영체제와 유사한 환경 제공

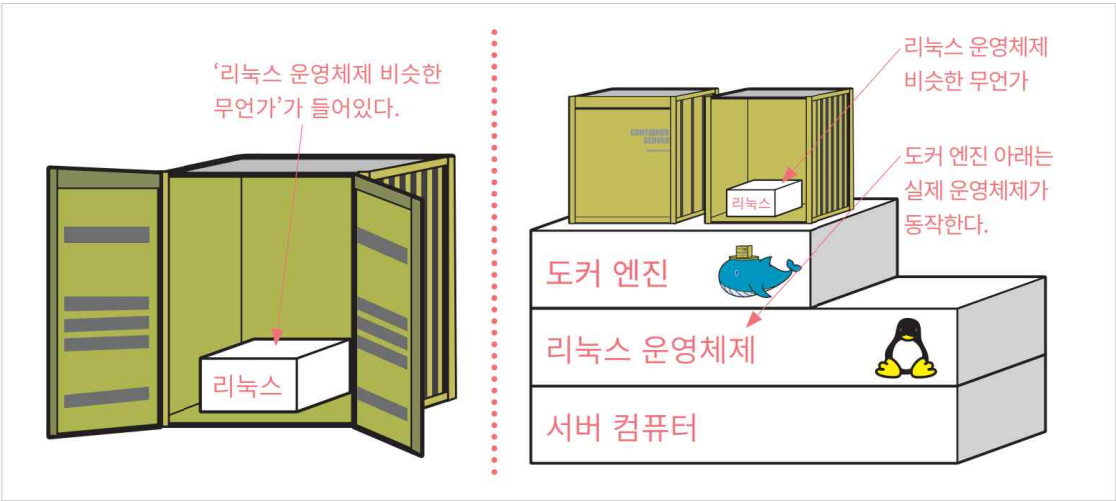


그림 2-1-2 컨테이너 속에는 '리눅스 운영체제 비슷한 무언가'가 들어 있다.

1.도커의 구조

• 운영체제의 구조

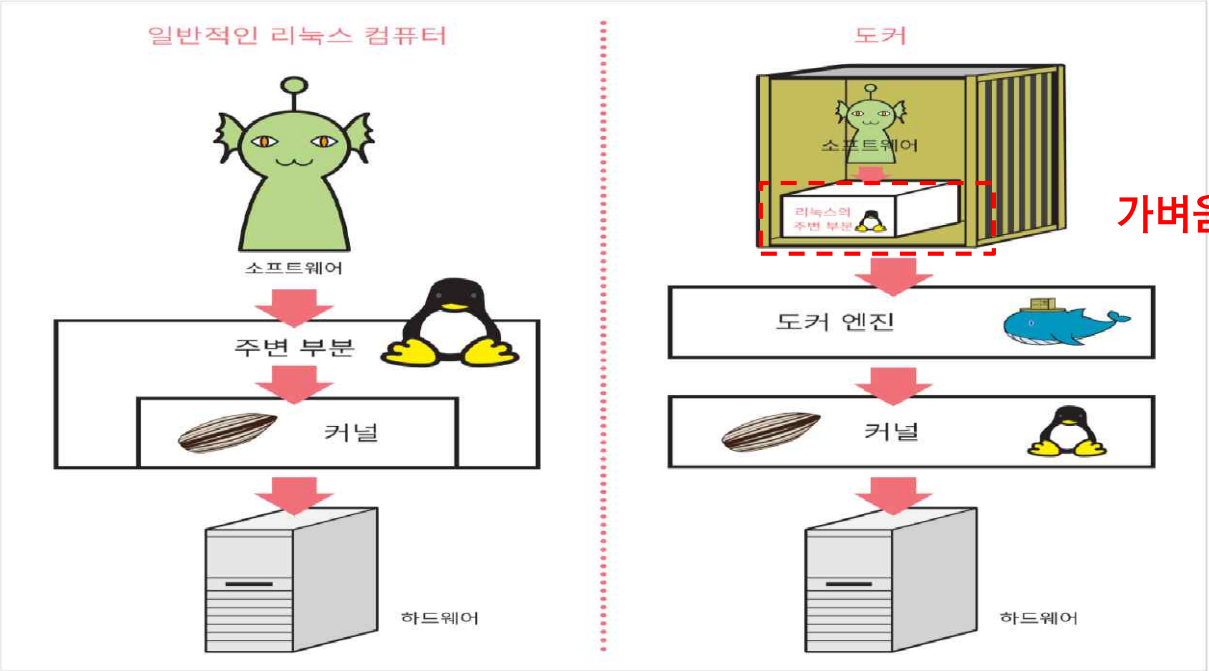


그림 2-1-4 컨테이너는 외부와 완전히 분리돼 있으므로 컨테이너 속에 운영체제의 주변 부분이 따로 들어 있다.

2.리눅스용 도커

- 도커는 기본적으로 리눅스 운영체제에서만 동작
- 컨테이너에서 실행할 소프트웨어 역시 리눅스용 소프트웨어
- 즉, 도커는 리눅스 컴퓨터에 독립된 격리 환경을 만드는 것

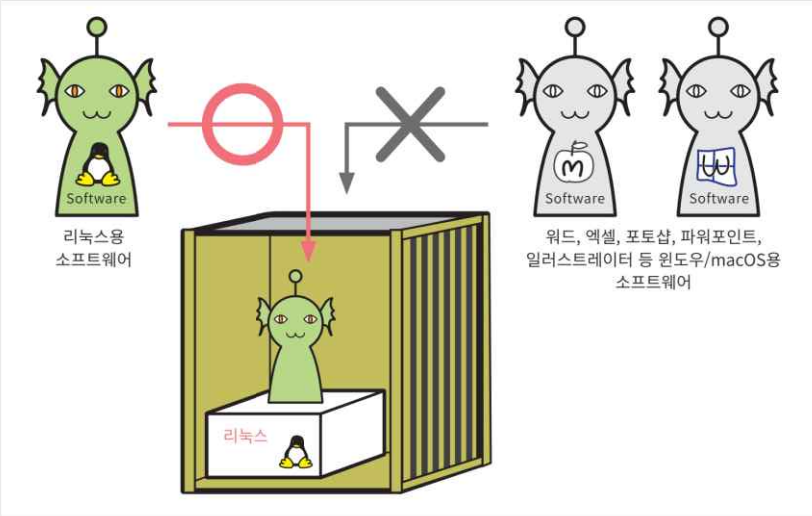


그림 2-1-5 컨테이너에 들어가는 프로그램은 리눅스용이어야 한다.

2.리눅스용 도커

- 윈도우와 macOS에서 도커 구동
- 윈도우나 macOS위에서 도커를 수행하려면 리눅스 운영체제를 갖추어야 함.

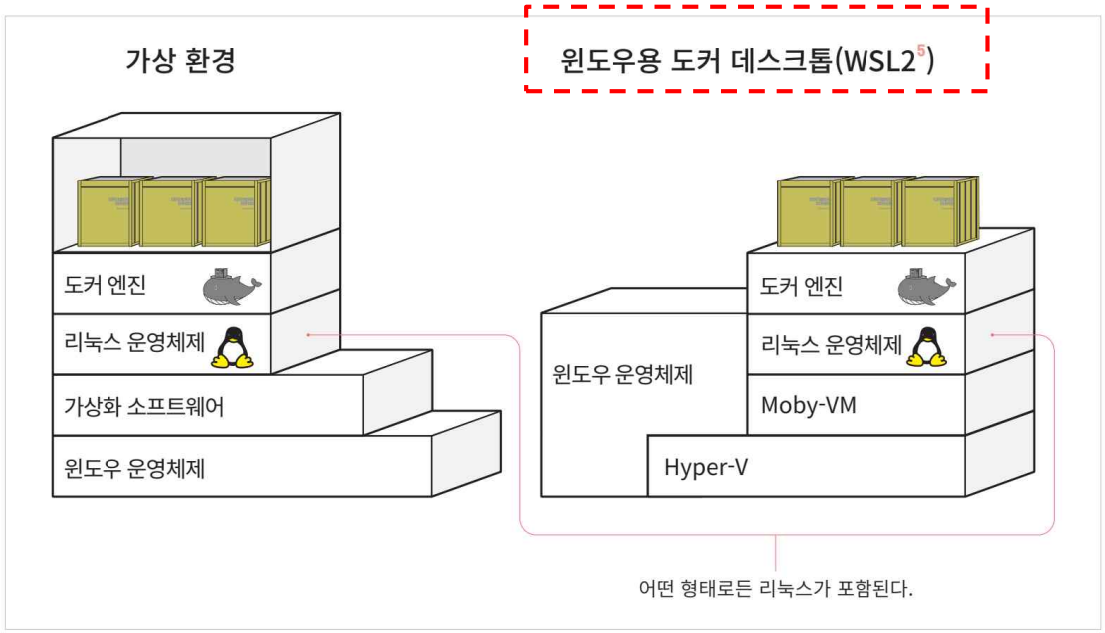


그림 2-1-6 도커는 기본적으로 리눅스용이다.

2장 도커의 동작 원리

2절 도커 허브와 이미지, 그리고 컨테이너

1.이미지와 컨테이너

• 이미지 : 컨테이너의 설계도

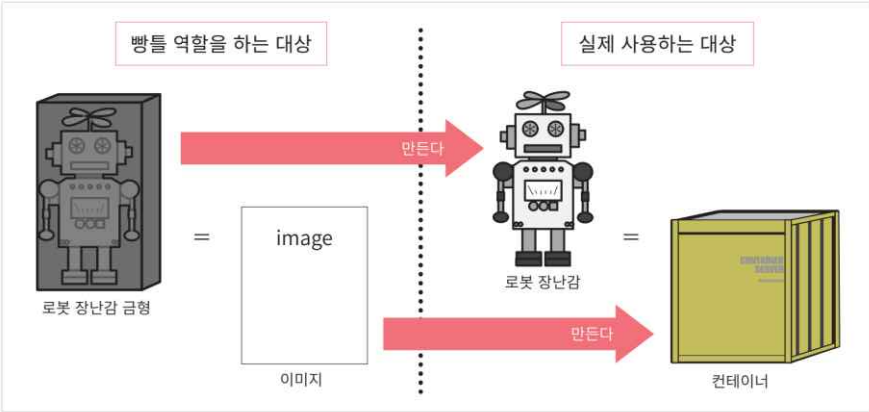


그림 2-2-1 이미지는 컨테이너를 만드는 데 사용한다.

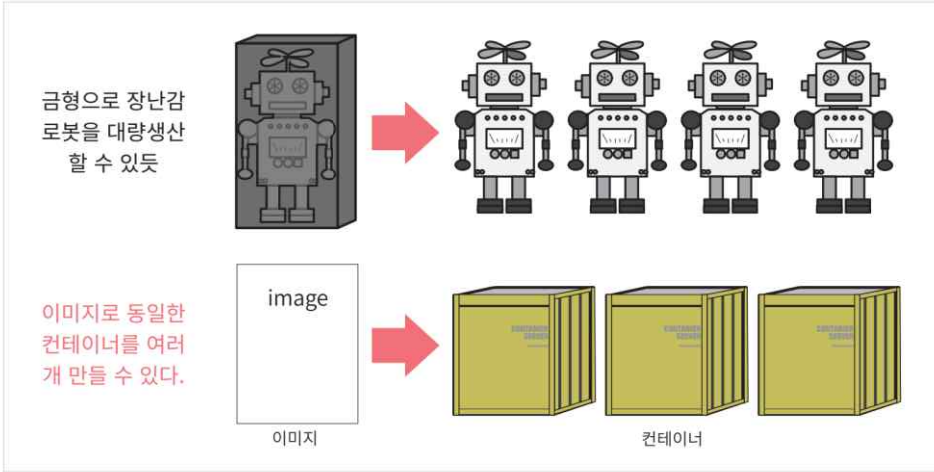


그림 2-2-2 하나의 이미지로 동일한 컨테이너를 여러 개 만들 수 있다.

1.이미지와 컨테이너

- 컨테이너로 이미지를 만들 수 있다.



그림 2-2-4 새로운 이미지를 사용해 '개조된' 컨테이너를 대량으로 만들 수 있다.

- 도커 엔진간 이동이 가능하다

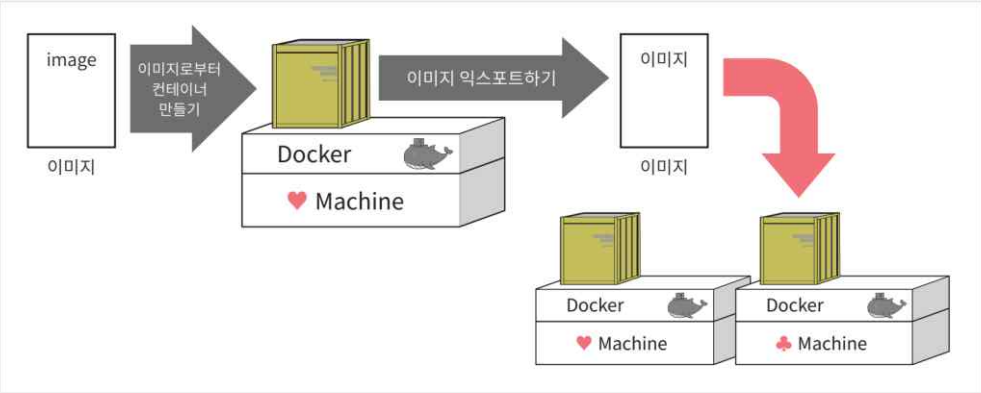


그림 2-2-5 컨테이너로부터 만든 이미지를 이용하면 다른 환경에서 원래의 컨테이너를 복원할 수 있다.

2.도커 허브와 도커 이미지

- 도커 허브(https://hub.docker.com)
  - 도커 레지스터리(이미지를 배포하는 사이트)
  - 공개된 컨테이너 이미지가 모여있는 사이트

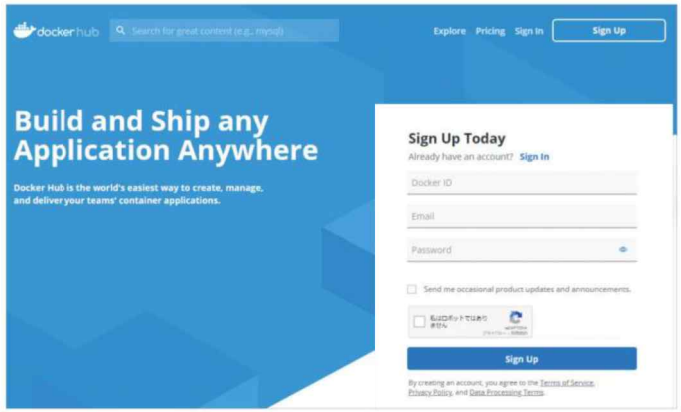
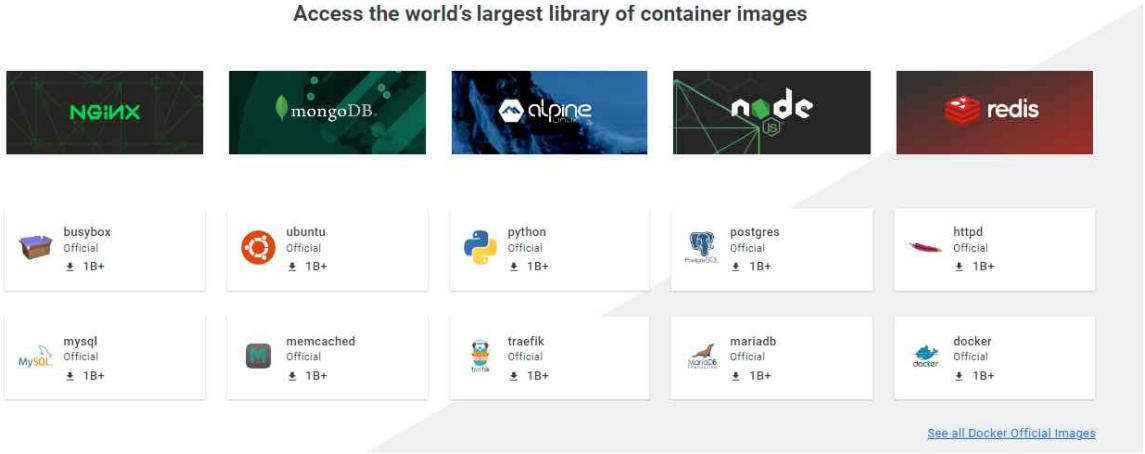


그림 2-2-6 도커 허브



2.도커 허브와 도커 이미지

• 도커허브에서 제공하는 이미지

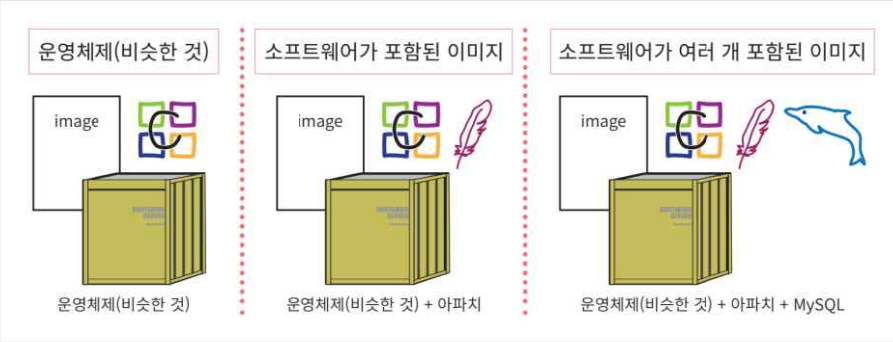


그림 2-2-8 도커 허브에는 다양한 이미지가 제공된다.

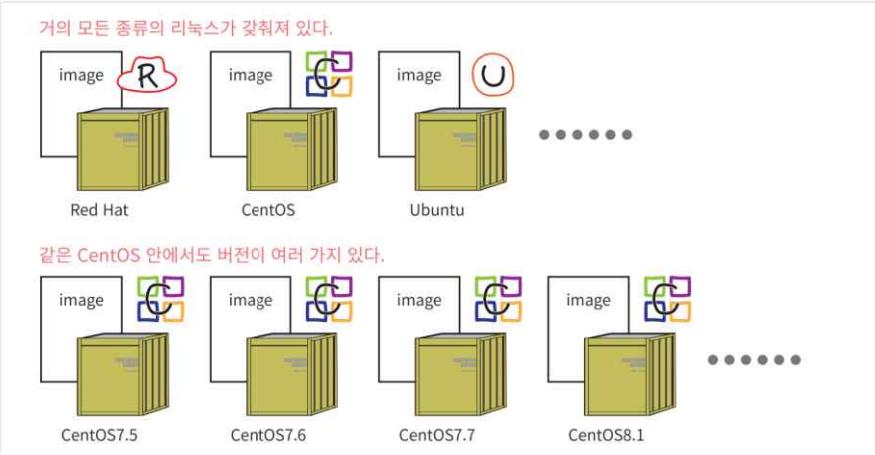


그림 2-2-9 운영체제(비슷한 것)만 들어있는 컨테이너도 여러 가지가 존재한다.

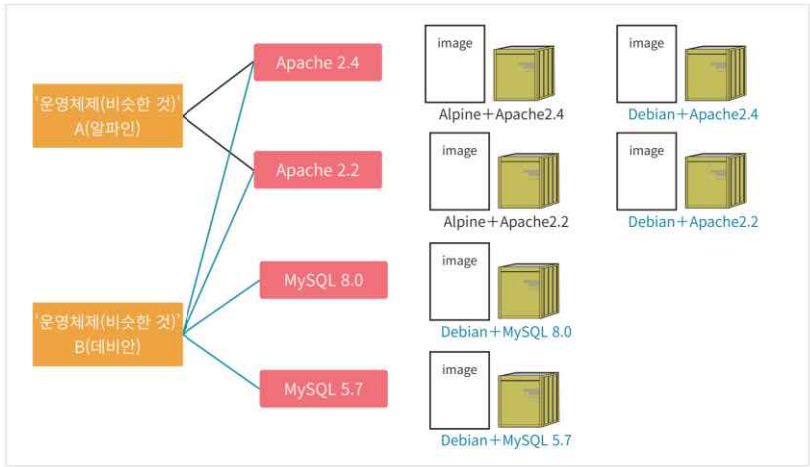


그림 2-2-10 '운영체제(비슷한 것)'과 소프트웨어의 조합은 버전에 따라 더 다양해진다.

2.도커 허브와 도커 이미지

- 안전한 컨테이너 이미지 고르는 방법
  - 공식 이미지를 사용한다
  - 커스텀 이미지를 직접 만들어 사용한다.

- 다양한 형태로 조합이 가능한 컨테이너
  - 도커 사용 원칙 '한 컨테이너에 한 프로그램'
  - 보안 및 유지 관리 측면에서 유리
  - 한 컨테이너에 필요한 프로그램을 모두 설치해도 가능

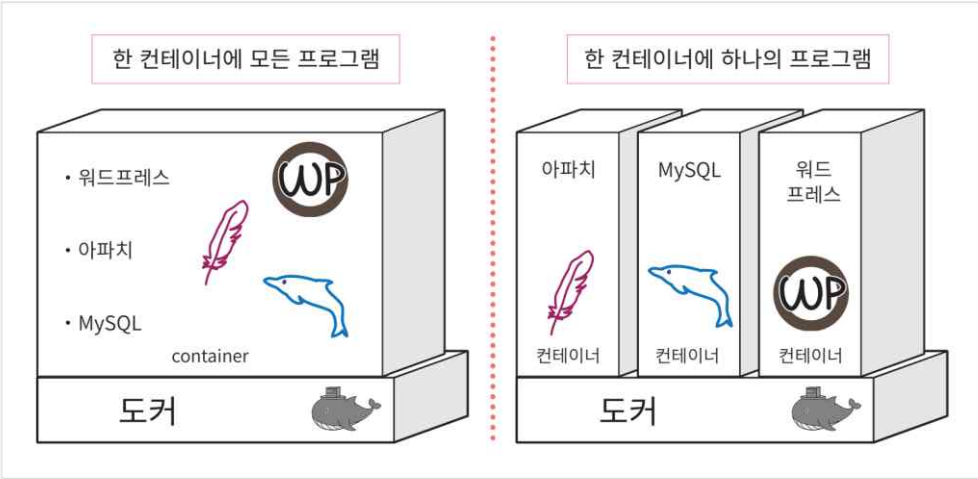


그림 2-2-11 한 컨테이너에 하나의 프로그램 또는 여러 개의 프로그램을 넣을 수도 있다.

2장 도커의 동작원리

3절 도커 컨테이너의 생애주기와 데이터 저장

1.도커 컨테이너의 생애주기와 데이터 저장

- 도커 컨테이너는 **일회용품**으로 생각

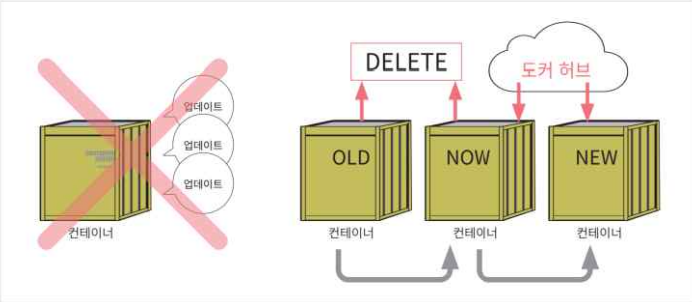


그림 2-3-1 컨테이너는 '쓰고 버리는' 일회용품에 가깝다.

- 컨테이너는 일반적으로 여러 개를 동시에 가동하는 경우가 많으므로 유지보수가 용이하지 않을 수 있다.
- 따라서 업데이트시 새로운 이미지로부터 새로운 컨테이너를 만들어 배포하는 방식을 이용

컨테이너의 생애주기

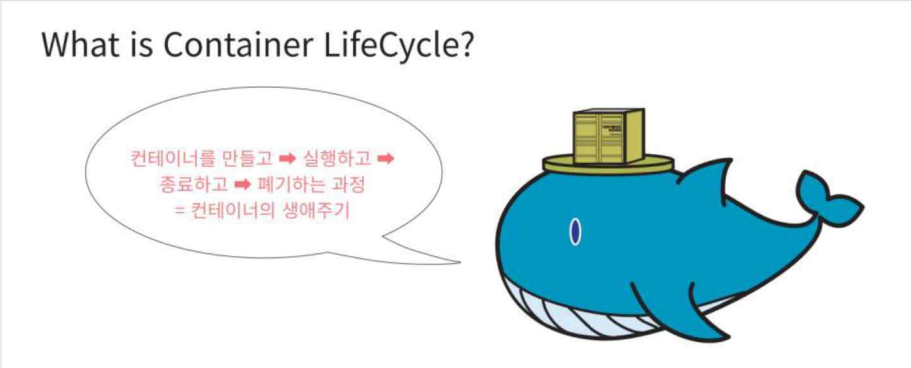


그림 2-3-2 컨테이너의 생애주기

2.데이터 저장

- 컨테이너 삭제시 그 안에 있던 파일도 함께 삭제
- 이를 방지하기위해 일반적으로 물리적 서버의 디스크를 마운트해 저장

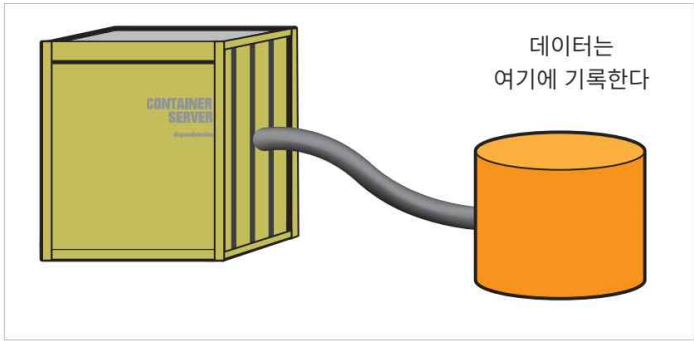


그림 2-3-3 물리적 컴퓨터의 일부를 마운트하고 여기에 데이터를 저장한다.

- 컨테이너를 삭제해도 데이터를 외부에 안전하게 보관
- 도커 엔진에 장애가 발생해도 데이터는 그대로 보존

2.데이터 저장

• 데이터의 외부 저장

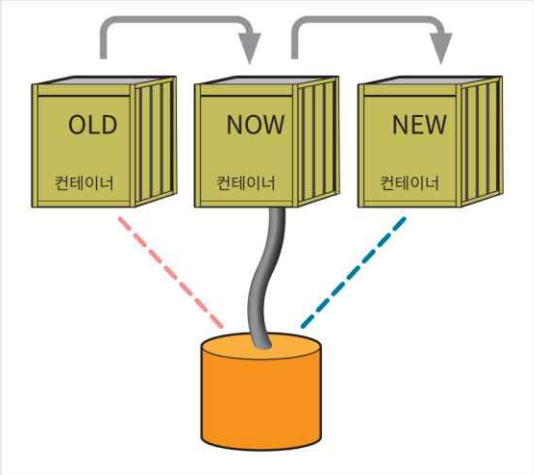


그림 2-3-4 컨테이너가 바뀌어도 외부에 저장된 데이터는 그대로 사용할 수 있다.

- 컨테이너가 바뀌어도 데이터는 계속 이용할 수 있다.
- 데이터나 설정 파일등은 외부에 저장
- 프로그램 개발시 외부에 저장하지않는 경우도 있으니 컨테이너 삭제시 확인 필요

2장 도커의 동작 원리

4절 도커의 장점과 단점

1. 도커의 구조와 특성 및 장단점

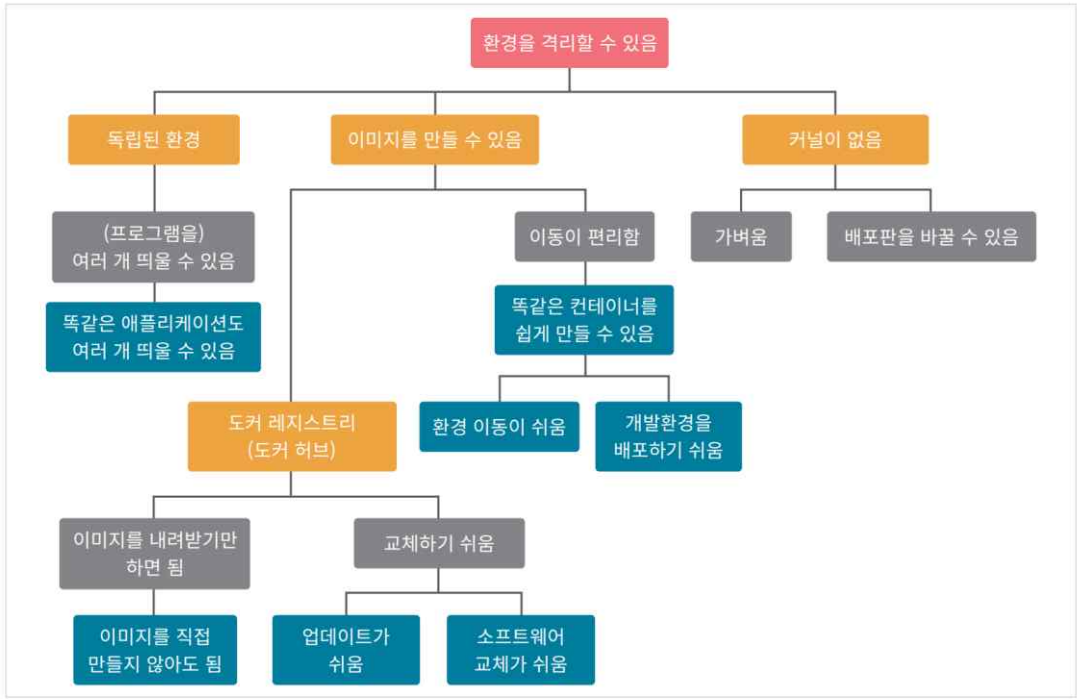


그림 2-4-1 도커의 성질

- 독립된 환경
- 이미지 생성 가능
- 컨테이너에 커널 포함되지않아 가벼움

⇨ 격리 가능

2.도커의 장점과 단점

- 장점
  - 1대의 물리적인 서버에 여러 대의 서버를 구동할 수 있다.
  - 서버 관리가 용이하다
    - 다른 소프트웨어에 영향을 미치지 않음
    - 컨테이너 교체나 수정이 쉬워 환경 이전도 수월
  - 서버 고수가 아니어도 다루기 수월
    - 명령 한줄로 서버 구축 완료

- 도커의 단점
  - 리눅스 소프트웨어만 지원
  - 호스트 서버에 문제발생시 모든 컨테이너에 영향을 미침
  - 컨테이너를 하나만 사용하는 경우에는 비효율적

3.도커의 주용도

- 팀원에게 동일한 개발환경 제공
  - 같은 프로젝트 팀원에게 개발환경을 동일하게 제공
  - 프로젝트별 컨테이너를 따로 사용할 수 있음
  - 개발환경과 운영환경의 차이가 없음

- 새로운 버전의 테스트
  - 새로운 운영체제나 라이브러리 버전을 개발환경에서 먼저 테스트하고 운영환경에 컨테이너로 적용

- 동일한 서버가 여러 대 필요한 경우
  - 1대의 물리 서버에 여러 개의 컨테이너를 이용할 수 있으므로 비용 절약
  - 명령 한 줄로 원하는 만큼 서버 구동 가능
  - 스케일링에 유리하므로 웹 서버나 API 서버로 많이 활용

3장 도커를 사용해보자

1절 도커 준비

1.도커는 모든 운영체제에서 사용 가능

• 도커 환경을 구축하는 방법

1. 리눅스 컴퓨터에서 도커 사용하기
2. 가상 머신이나 렌탈 환경에 도커를 설치하고 윈도우나 macOS를 통해 사용하기
3. 윈도우용/macOS용 도커 사용하기

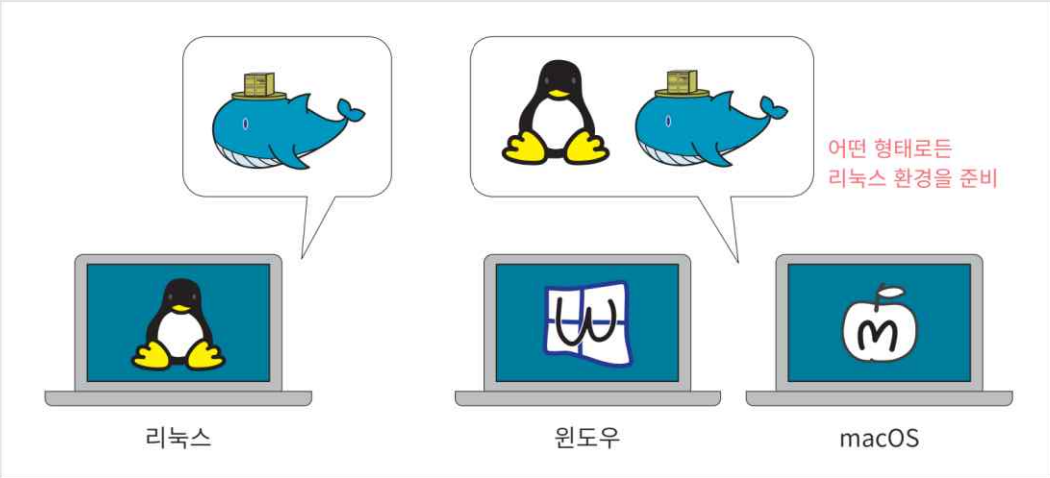


그림 3-1-1 리눅스, 윈도우, macOS 각각의 환경에 맞춰 도커 환경을 준비한다.

명령어는 공통적으로 사용

2.윈도우용/macOS용 도커 사용하기

- 리눅스 운영체제가 들어있는 패키지, '도커 데스크톱' 이용

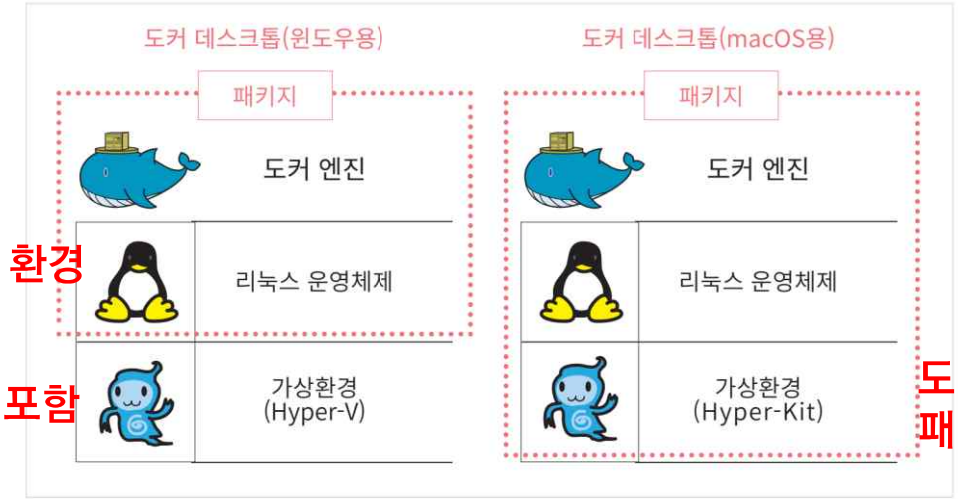


그림 3-1-2 도커 데스크톱에는 도커 엔진 외에 실행환경이 함께 패키징돼 있다.

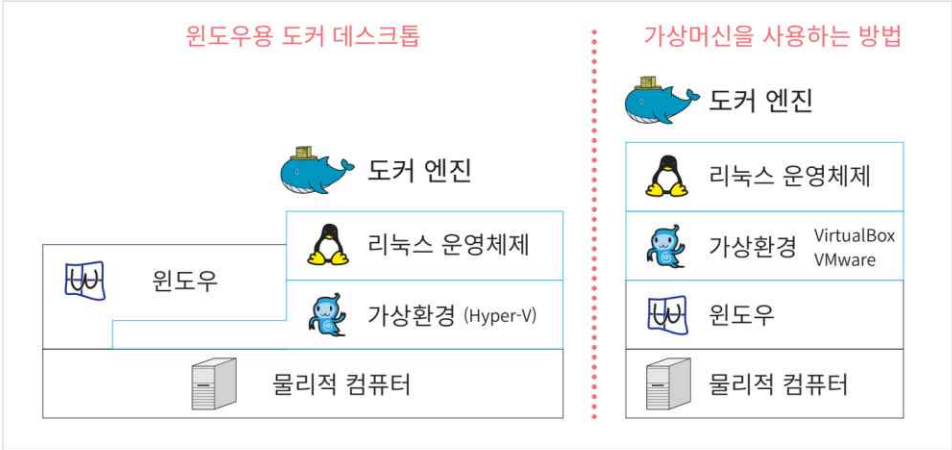


그림 3-1-3 도커 데스크톱과 가상화 소프트웨어를 사용한 방법의 차이점

2.윈도우용/macOS용 도커 사용하기

- 도커 데스크톱 일반 소프트웨어처럼 사용
- 도커 엔진을 조작할 때는 CLI(Command Line Interface) 방식 이용
- 리눅스는 도커 전용 OS로만 사용

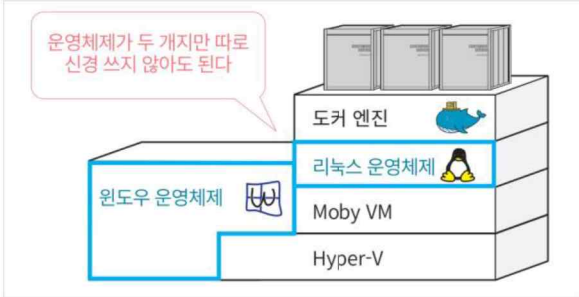


그림 3-1-4 도커 데스크톱을 사용할 때 운영체제가 두 개라는 점에 신경 쓰지 않아도 된다(그림은 윈도우용 도커 데스크톱과 뒤에 설명할 WSL2의 조합을 나타낸다).

- 윈도우용 가상환경(Hyper-V) 이용시 속도가 느려지거나 다른 가상화 SW와 충돌 발생 경우가 있음
- macOS에서는 약간 무겁게 느껴짐, M1,M2 칩으로 전환되면서 안정화 기간 필요

2. 윈도우용/macOS용 도커 사용하기

· 도커 사전 준비

윈도우용/macOS용 도커 데스크톱을 사용하기 위해 필요한 것

· 사용 조건을 만족하는 윈도우 또는 macOS

주의할 점

· 평소에 가상화 소프트웨어를 사용했다면 윈도우와 가상화 소프트웨어를 모두 최신 버전으로 업데이트해야 한다.

교재 55페이지 WSL2(Windows Subsystem for Linux2) COLUMN 참조

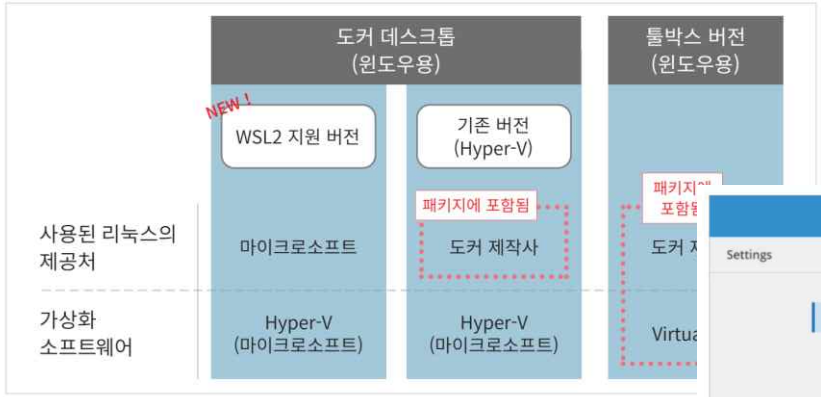


그림 3-1-6 도커 데스크톱의 두 가지 종류

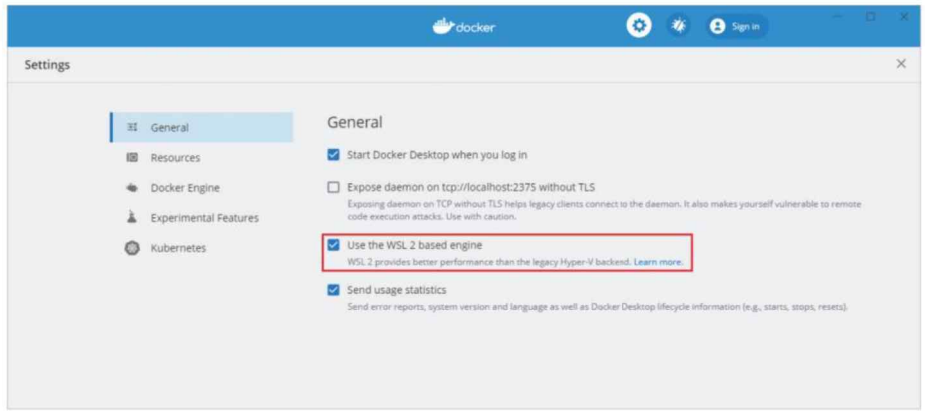


그림 3-1-7 도커 데스크톱의 설정(Settings) 화면(이 화면을 띄우는 방법은 3.2절을 참조)

3.도커 실행 조건

- 도커엔진 설치 조건
- 64비트 운영체제
- 윈도우
  - 윈도우10 64비트 버 이후 버전)
  - 윈도우10 64비트 버
  - Hyper-V 및 Cont
  - CPU : SLAT 기능을
  - 메모리 4GB 이상
  - BIOS에서 Virtualization 활성화
- masOS
  - 2010년 이후 발매된 모델
  - macOS 10.13(하이 시에라)이후 버전
  - 메모리 4GB 이상

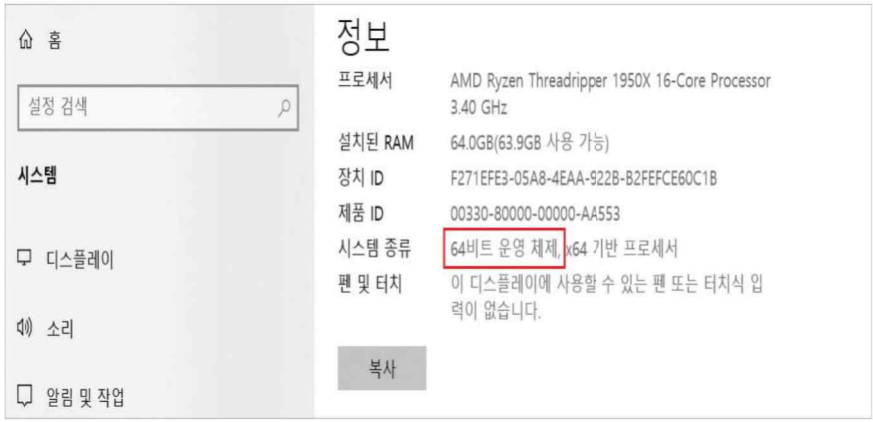


그림 3-1-9 시스템 정보 확인

3.도커 실행 조건

- 리눅스
  - 배포판 및 버전

지원하는 리눅스 배포본 및 버전

배포본	버전
CentOS	CentOS 7 이후
우분투	우분투 16.04 이후
데비안	데비안 9(스트레치) 이후
페도라	페도라 30 이후

  - 리눅스 커널 3.10 이후
  - Iptables 1.4 이후
  - Git 1.7 이후
  - XZ 도구 4.9 이후
  - procps 와 cgroup 계층 준수

3장 도커를 사용해보자

2절 도커 설치

부록2 macOS용 도커 설치

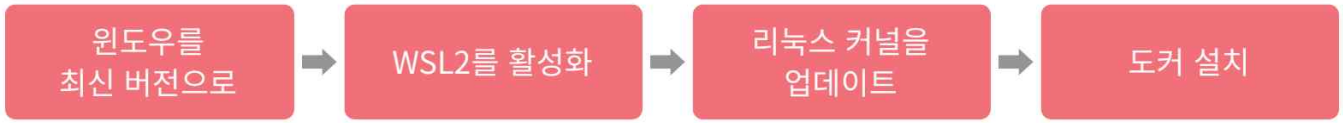
부록3 리눅스용 도커 설치

1.도커 설치

- 윈도우용 도커 데스크톱 설치

  WSL2 사용 기준 (윈도우 1903/19019 이후 버전) 부록 1 참조

- 설치 순서



1) 윈도우 최신버전으로 업데이트(1903/1909이후 버전 확인)
2) Windows 기능 켜기/끄기 창에서
   Linux용 Windows 하위시스템과 가상머신 플랫폼 체크

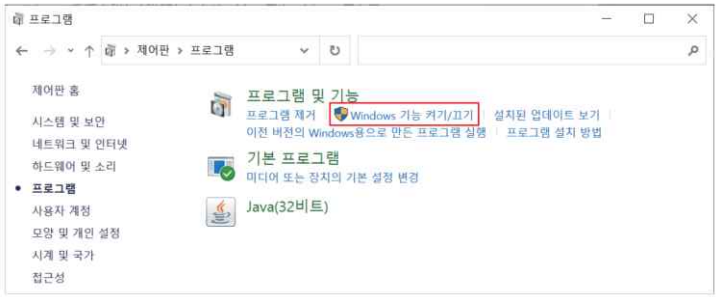


그림 3-2-3 [Windows 기능 켜기/끄기]를 클릭한다.

3) 컴퓨터 재기동

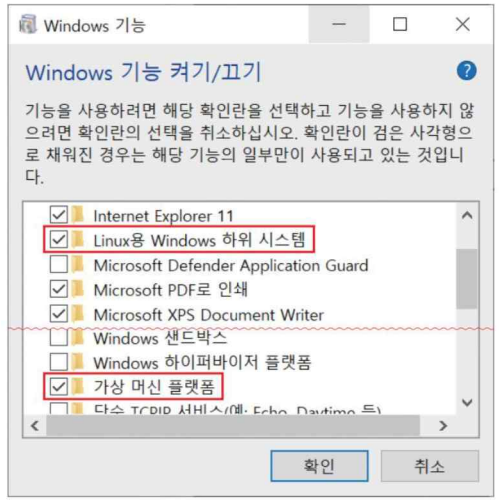


그림 3-2-4 두 가지 기능을 체크한다.

1. 도커 설치

4) 리눅스 커널 다운로드 및 업데이트

https://wslstorestorage.blob.core.windows.net/wslblob/wsl_update_x64.msi

다운로드 후 실행

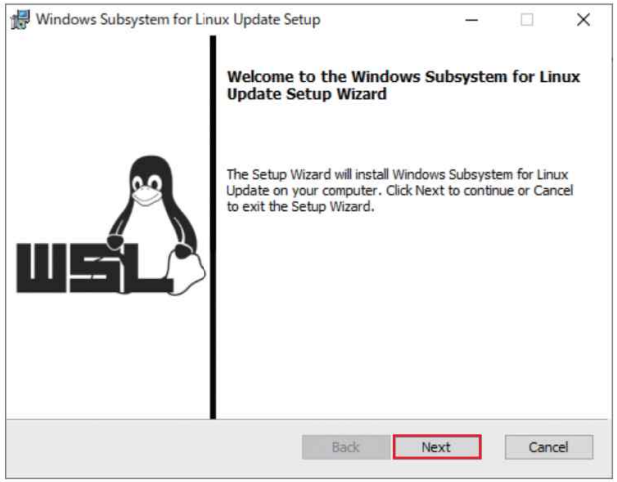


그림 3-2-6 리눅스 커널 업데이트

참고 : https://learn.microsoft.com/ko-kr/windows/wsl/install

1.도커 설치

5) 윈도우용 데스크톱 다운로드 및 설치

https://docs.docker.com/desktop/install/windows-install/





그림 3-2-8 설치 파일을 더블클릭해 실행

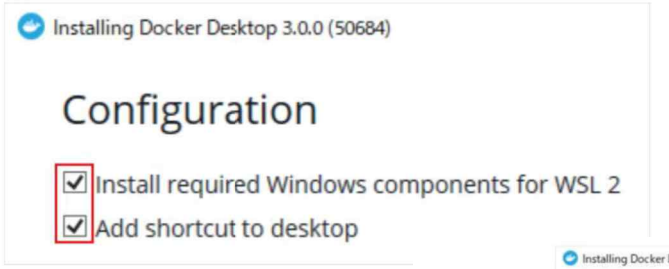


그림 3-2-9 윈도우용 도커 데스크톱의 설치 진행 화면

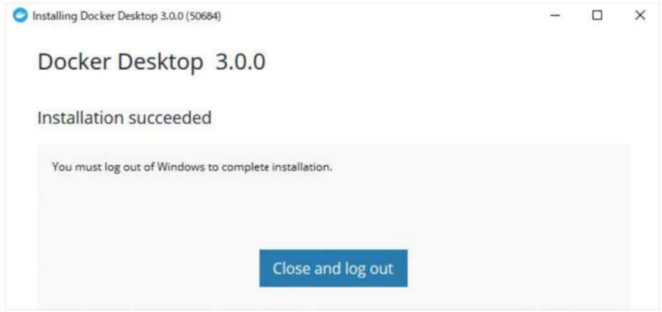
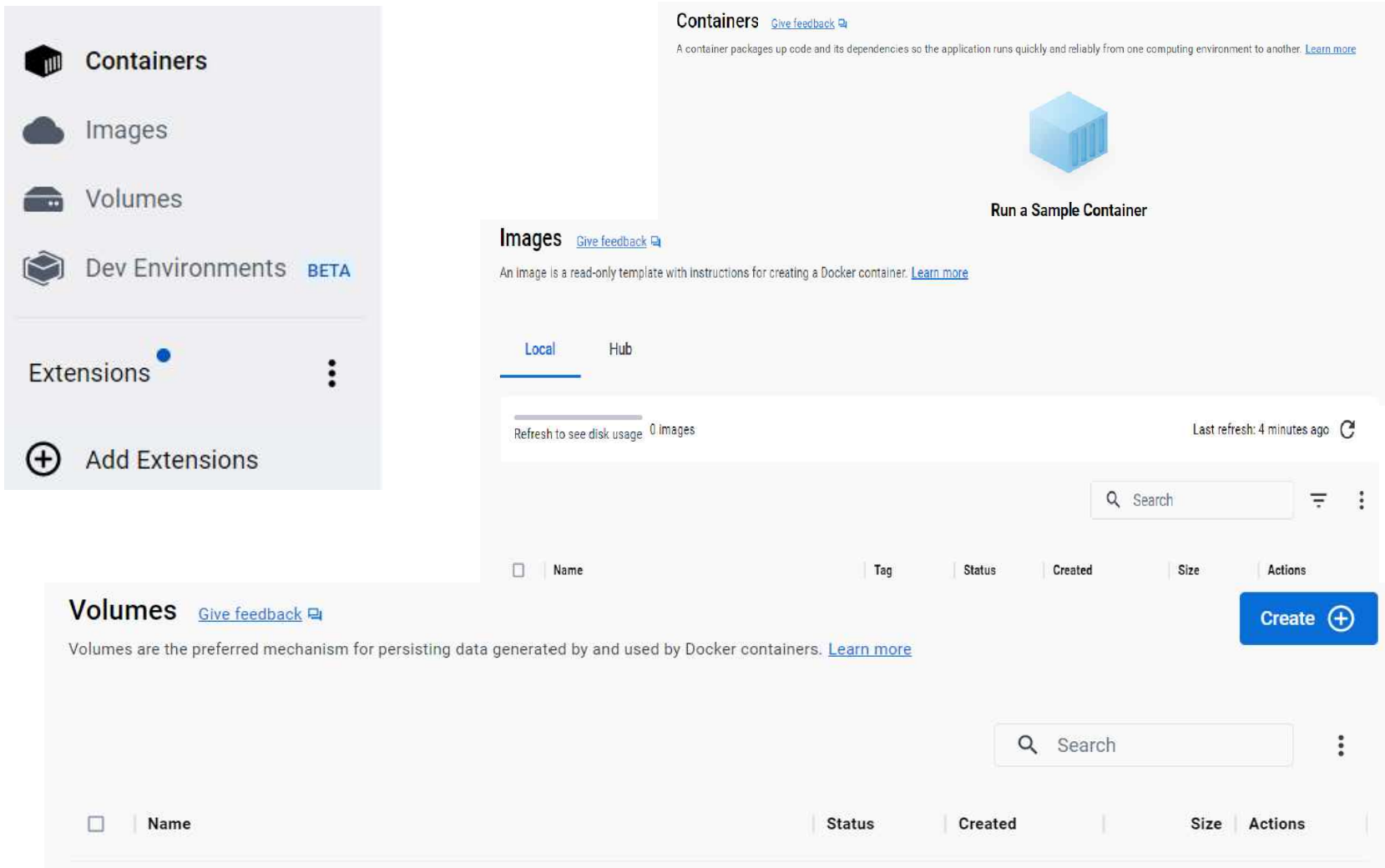


그림 3-2-10 설치 완료

2. 도커 데스크톱 화면

- 홈 화면



2. 도커 데스크톱 화면

• 도커 실행여부 (왼쪽 하단)

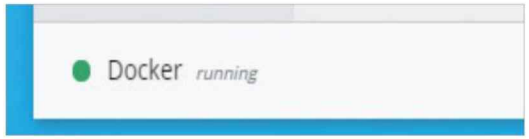


그림 3-2-16 정상 동작 중임을 나타내는 마크

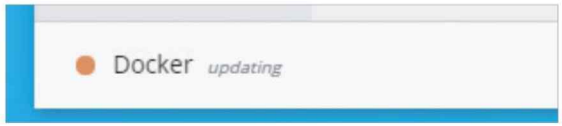
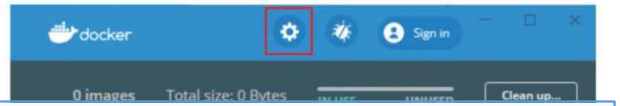
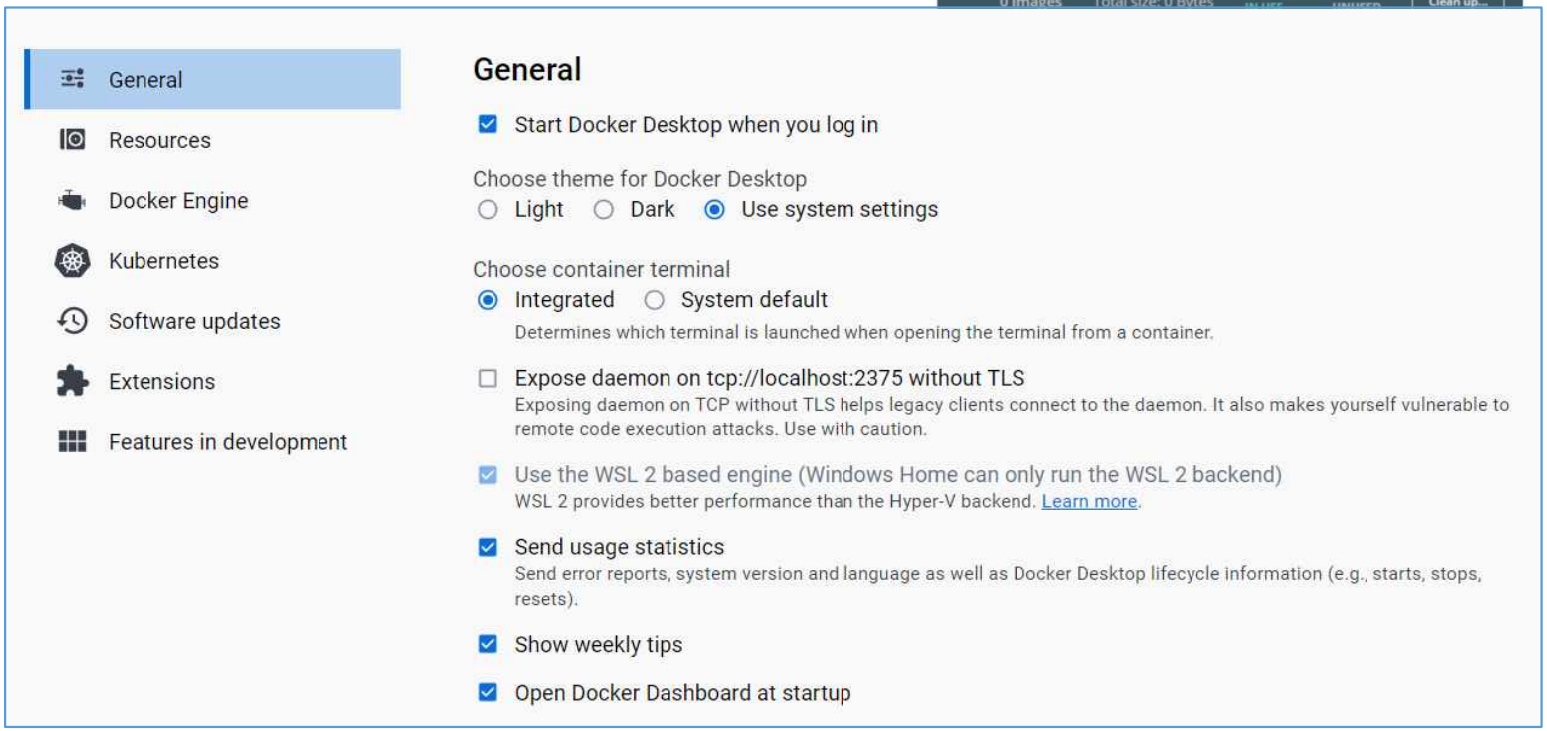


그림 3-2-17 업데이트 또는 중지 상태임을 나타내는 마크

• 도커 설정 화면 (오른쪽 상단)





2. 도커 데스크톱 화면

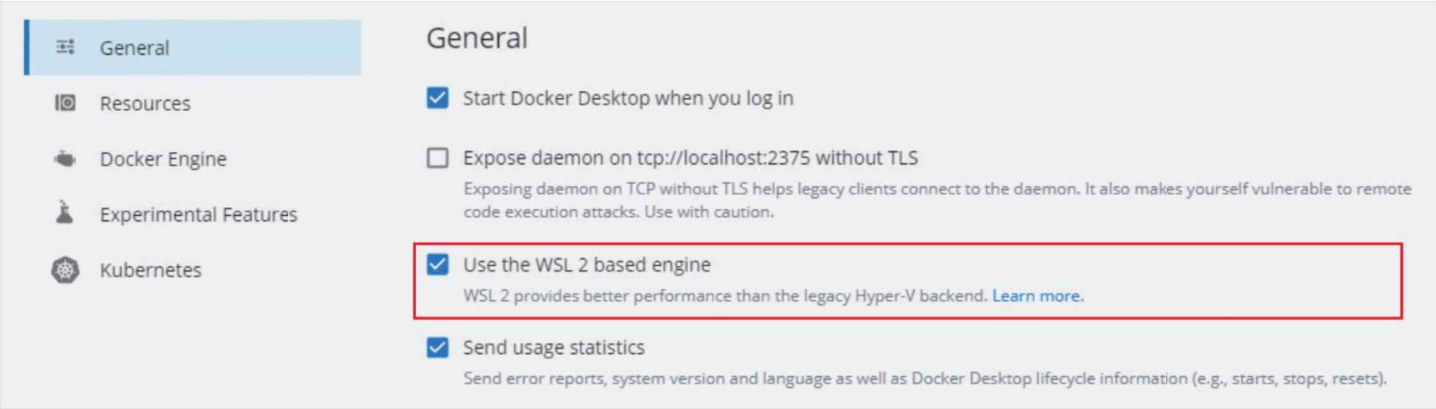


그림 3-2-19 도커 엔진이 WSL2에서 동작하는지 확인

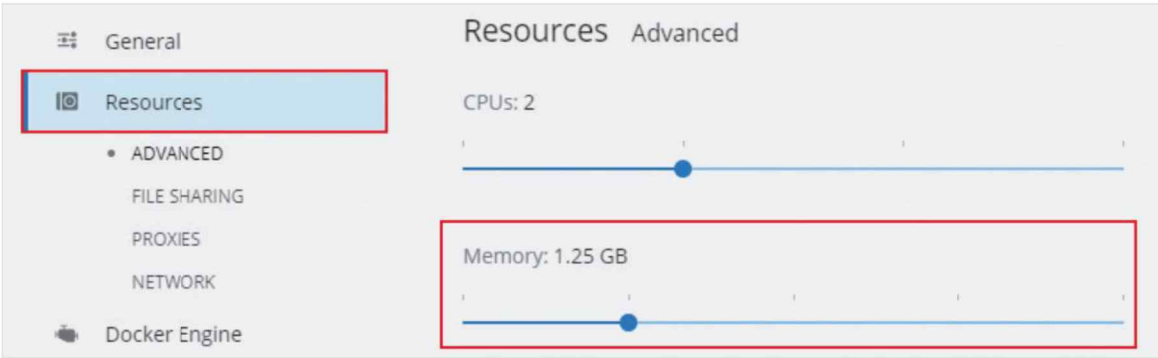


그림 3-2-20 'Resource' 탭에서 'Memory'의 설정값 조정

4장 컨테이너를 실행해보자

1절 도커 엔진 시작하기와 종료하기

1.도커 엔진 시작/종료

- 도커 엔진 자동 실행
  - 컨테이너를 실행하지않으면 컴퓨터의 리소스는 거의 차지하지 않음
- 도커엔진이 종료되면 컨테이너는 모두 정지됨
- 도커엔진은 자동으로 구동되나 컨테이너는 자동으로 구동되지 않음

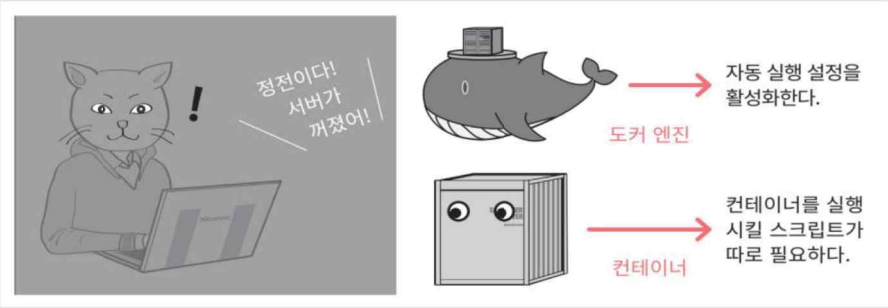


그림 4-1-1 도커 엔진은 자동 실행 설정이 가능하지만 컨테이너는 그렇지 않다.

OS	도커엔진 시작	도커엔진 종료	자동실행설정
윈도우	시작 → Docker Desktop 선택	도커 데스크톱 창 닫기	기본으로 자동 구동
macOS	파인더 → '애플리케이션' 폴더 → Docker Desktop 선택	상태막대에서 도커 클릭 -> Quit Docker Desktop 선택	상태막대에서 도커 클릭 → Preference → Start Docker Desktop when you login 선택
리눅스	sudo systemctl start docker	sudo systemctl stop docker	sudo systemctl enable docker

4장 컨테이너를 실행해보자

2절 컨테이너의 기본적인 사용방법

1.도커 기본 명령어

- 도커 명령의 구성

docker 상위명령 하위명령 [옵션] 대상 [인자]

상위명령
- container
- image
- .
- .
- .

하위명령
- run
- pull
- start
- .
- .

start/stop/run 명령인 경우 container 생략 가능

[옵션]
명령의 세부 설정 시 이용
-d : 백그라운드 실행
-i or -t : 대화형 실행

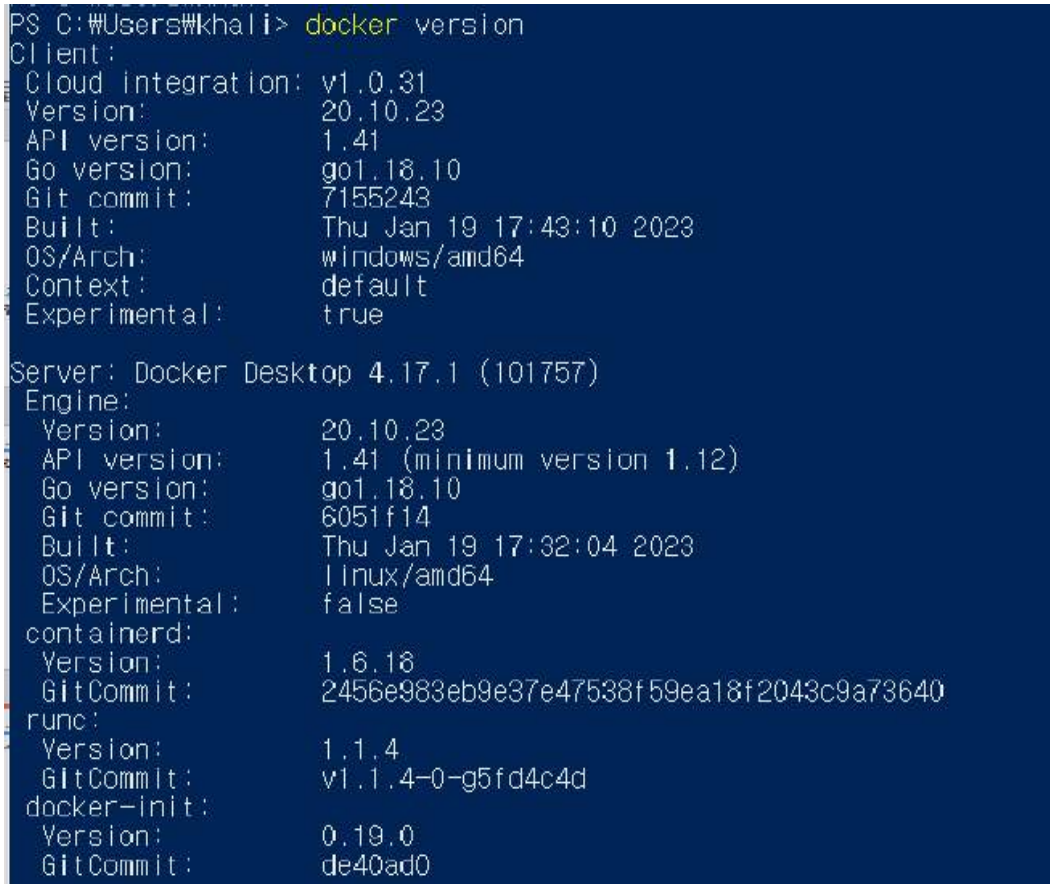
대상
명령을 수행할 구체적인 대상

[인자]
대상에 전달할 값 지정

1.도커 기본 명령어

<실습> 버전 확인

```
docker version
```



2.대표적인 docker 명령

- 컨테이너를 다루는 도커 명령

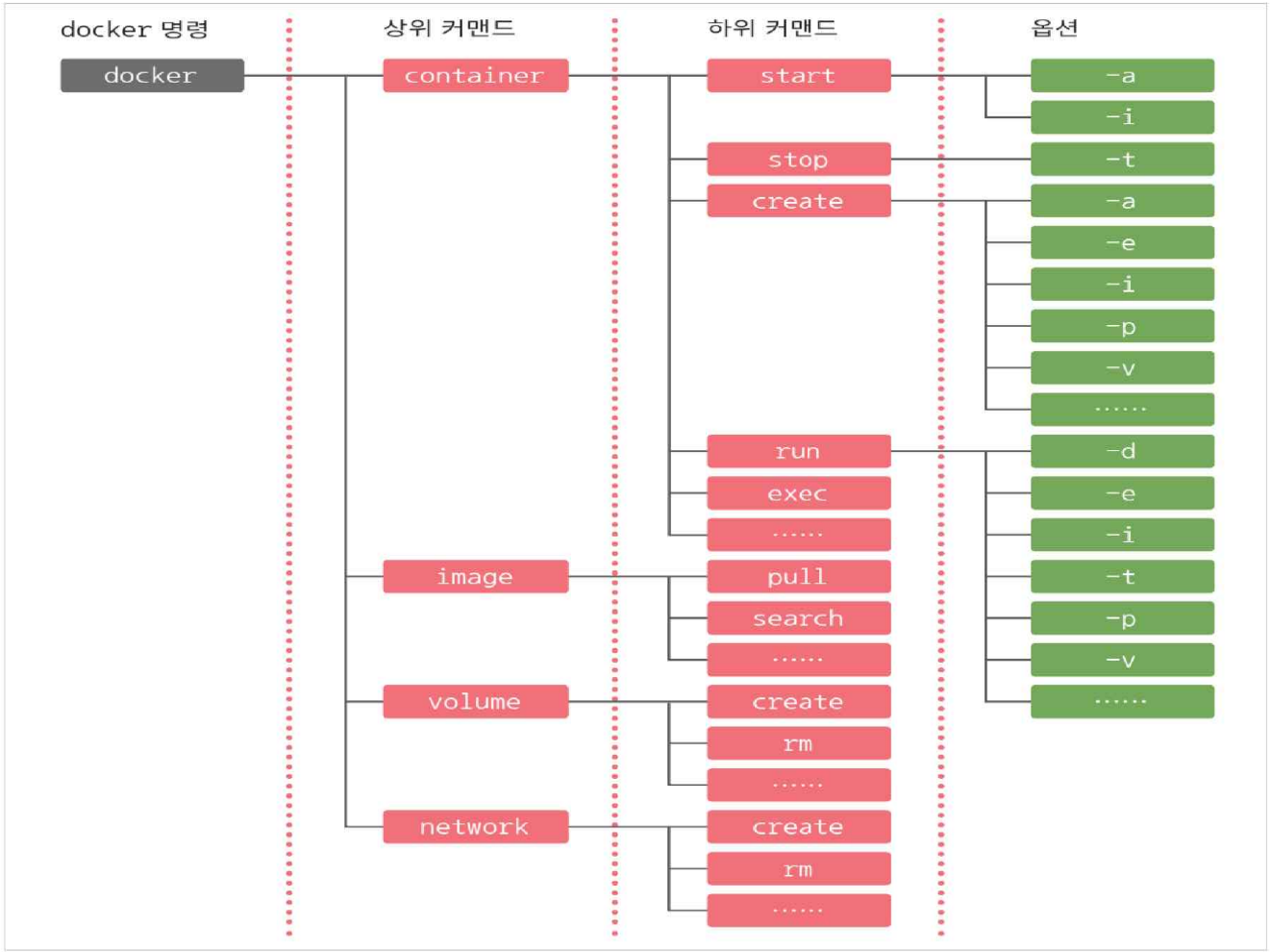


그림 4-2-1 대표적인 명령어

2.대표적인 docker 명령

• 컨테이너 제어 명령

`docker [container] 하위명령 옵션`

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
start	컨테이너를 실행	O	-i
stop	컨테이너를 정지	O	거의 사용하지 않음
create	도커 이미지로부터 컨테이너를 생성	O	--name -e -p -v
run	도커 이미지를 내려받고 컨테이너를 생성해 실행함(다운로드는 필요한 경우에만). docker image pull, docker container create, docker container start라는 세 개의 명령을 하나로 합친 것과 같다.	O	--name -e -p -v -d -i -t
rm	정지 상태의 컨테이너를 삭제	O	-f -v
exec	실행 중인 컨테이너 속에서 프로그램을 실행	O	-i -t
ls	컨테이너 목록을 출력	*1	-a
cp	도커 컨테이너와 도커 호스트 간에 파일을 복사	O	거의 사용하지 않음
commit	도커 컨테이너를 이미지로 변환	O	거의 사용하지 않음

*1: 생략형은 docker ps

2.대표적인 docker 명령

• 이미지 제어 명령

docker image 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
pull	도커 허브 등의 리포지토리에서 이미지를 내려받음	O	거의 사용하지 않음
rm	도커 이미지를 삭제	*2	거의 사용하지 않음
ls	내려 받은 이미지의 목록을 출력	X	거의 사용하지 않음
build	도커 이미지를 생성	O	-t

*2: 생략형은 docker rmi

2.대표적인 docker 명령

• 볼륨 제어 명령

```
docker volume 하위명령   옵션
```

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
create	볼륨을 생성	X	--name
inspect	볼륨의 상세 정보를 출력	X	거의 사용하지 않음
ls	볼륨의 목록을 출력	X	-a
prune	현재 마운트되지 않은 볼륨을 모두 삭제	X	거의 사용하지 않음
rm	지정한 볼륨을 삭제	X	거의 사용하지 않음

2.대표적인 docker 명령

• 네트워크 제어 명령

```
docker network 하위명령어   옵션
```

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
connect	컨테이너를 도커 네트워크에 연결	X	거의 사용하지 않음
disconnect	컨테이너의 도커 네트워크 연결을 해제	X	거의 사용하지 않음
create	도커 네트워크를 생성	X	거의 사용하지 않음
inspect	도커 네트워크의 상세 정보를 출력	X	거의 사용하지 않음
ls	도커 네트워크의 목록을 출력	X	거의 사용하지 않음
prune	현재 컨테이너가 접속하지 않은 네트워크를 모두 삭제	X	거의 사용하지 않음
rm	지정한 네트워크를 삭제	X	거의 사용하지 않음

2.대표적인 docker 명령

• 기타 명령어

그 밖의 상위 커맨드

상위 커맨드	내용
checkpoint	현재 상태를 일시적으로 저장한 후, 나중에 해당 시점의 상태로 되돌릴 수 있다. 현재 실험적 기능이다.
node	도커 스웜의 노드를 관리하는 기능
plugin	플러그인을 관리하는 기능
secret	도커 스웜의 비밀값 정보를 관리하는 기능
service	도커 스웜의 서비스를 관리하는 기능
stack	도커 스웜 또는 쿠버네티스에서 여러 개의 서비스를 합쳐 구성한 스택을 관리하는 기능
swarm	도커 스웜을 관리하는 기능
system	도커 엔진의 정보를 확인하는 기능

• 단독 명령어

단독 커맨드[9]

단독 커맨드	내용	주요 옵션
login	도커 레지스트리에 로그인	-u -p
logout	도커 레지스트리에 로그아웃	거의 사용하지 않음
search	도커 레지스트리를 검색	거의 사용하지 않음
version	도커 엔진 및 명령행 도구의 버전을 출력	거의 사용하지 않음